



RIACHUELO

## Case Técnico para Analista de Sistemas (Squad Pix)

### Implementação de uma API para Pix Recorrente Seguro

#### Contexto e Desafio

A Squad Pix está desenvolvendo a funcionalidade de **Pagamentos Pix Recorrentes** (assinaturas ou mensalidades automatizadas). O Analista Sênior será responsável por projetar e construir um **Protótipo de Prova de Conceito (PoC)** de uma API robusta que garanta a **automatização** e a **segurança** (prevenção a fraudes) das transações agendadas.

O desafio é **prototipar uma API RESTful** que gerencie o ciclo de vida do agendamento e simule a orquestração segura de cada pagamento.

---

### 1. Requisitos Obrigatórios (Escopo do Projeto Prático)

O candidato deve criar um projeto funcional (PoC) usando as linguagens Java ou Node.js e publicá-lo em um repositório Git (GitHub, GitLab, etc.).

#### 1.1. Estrutura e API

1. **Endpoints RESTful:** Implementar **pelo menos dois endpoints** funcionais:
  - o `POST /v1/agendamentos`: Para registrar um novo agendamento recorrente.
  - o `GET /v1/agendamentos/{id}`: Para consultar o status de um agendamento.
2. **Modelo de Dados Simplificado:** Usar classes/estruturas simples para simular as entidades principais (`Agendamento`, `PagamentoRecorrente`) e persistir os dados em memória ou em um banco de dados simples (Ex: SQLite, arquivo JSON, ou simulação de repositório).

#### 1.2. Módulo de Prevenção a Fraudes (Simulação)

O projeto deve incluir uma função/serviço que simule o Módulo de Fraude.

1. **Integração:** O *endpoint* de agendamento deve chamar este Módulo de Fraude **antes** de aceitar a recorrência.
2. **Lógica de Decisão:** O módulo deve implementar **pelo menos 3 regras de negócio** simples que retornem um Status de Risco (Ex: APROVADO, REJEITADO, REVISAO\_MANUAL) com base nos dados de entrada.
3. **Registro:** O resultado da análise de risco deve ser registrado (em log ou no objeto de agendamento).

### 1.3. Documentação Técnica

O projeto no Git deve conter um arquivo **README.md** claro, explicando:

- Como rodar o projeto.
  - A URL dos *endpoints* implementados.
  - A **justificativa** para as regras de fraude criadas.
- 

## 2. Entrega Final (Git e Vídeo)

A entrega deve ser composta por dois itens, que serão a base da avaliação:

### 2.1. Projeto de Código Fonte (Git)

- O repositório Git deve estar **público** para acesso.
- Deve conter o código-fonte da PoC da API, com as funcionalidades acima.
- O histórico de *commits* será avaliado quanto à organização e clareza.

### 2.2. Vídeo de Apresentação e Justificativa (Máx. 5 minutos)

O vídeo deve ser conciso e focado, abordando os seguintes pontos:

1. **Demonstração:** Mostrar o projeto rodando, chamando um ou dois *endpoints* e o resultado.
  2. **Visão:**
    - **Decisões de Design:** Por que escolheu essa linguagem/estrutura?
    - **Resiliência:** Como sua API trataria **falhas** (Ex: *retry*, idempotência) em um ambiente de produção real.
    - **Trade-offs:** Qual o principal compromisso (Ex: Performance vs. Segurança) que você fez e por quê.
  3. **Fraude:** Explicar a lógica das regras de fraude implementadas e como elas protegem a squad de Pix.
-

### 3. Critérios de Avaliação

| Critério                        | Foco da Avaliação                                                       | Nível Esperado                                                                            |
|---------------------------------|-------------------------------------------------------------------------|-------------------------------------------------------------------------------------------|
| <b>Arquitetura &amp; Código</b> | Qualidade do código, design da API, clareza do <code>README.md</code> . | Modularidade, uso de <i>patterns</i> adequados (Mesmo que simples), clareza e manutenção. |
| <b>Prevenção à Fraude</b>       | Relevância e eficácia das regras de negócio implementadas.              | Pensamento estratégico sobre os riscos de um produto de Pix recorrente.                   |
| <b>Resiliência</b>              | Capacidade de prever e mitigar falhas em ambiente de produção.          | Proposição de soluções robustas (Filas, DLQ, Idempotência) e visão sistêmica.             |
| <b>Comunicação</b>              | Clareza, objetividade e poder de síntese no vídeo e na documentação.    | Capacidade de justificar decisões técnicas e traduzir o código em valor de negócio.       |